

数字电路设计第 6 章

数字构建模块

Digital Building Blocks (DDCA Ch5)

★★★★ 考试重点章节

考试重要性：★★★★

基于 Digital Design and Computer Architecture: ARM Edition (Harris & Harris)

2026 年 6 月 | 任胜兵老师考试重点整理

考试重点概览（任老师原话）

- ★★★★ ALU 标志位 (flag): ”曾经是考试题，下次还会考”——N/Z/C/V 四标志位含义和设置规则
 - ★★★★ 移位器三种类型: 逻辑移位 (Logical Shift)、算术移位 (Arithmetic Shift)、循环移位 (Rotate/Circular Shift)
 - ★★★★ IEEE 754 浮点数格式: ”等于送分给你们”——单精度格式 (符号位 1+ 阶码 8+ 尾数 23)、前导 1 隐藏原理、32 位浮点数加减运算
-

目录

1 加法器 (Adders)	5
1.1 1 位加法器 (1-Bit Adders)	5
1.1.1 半加法器 (Half Adder)	5
1.1.2 全加法器 (Full Adder)	5
1.2 多比特加法器——进位传播加法器 (CPA)	5
1.3 行波进位加法器 (Ripple-Carry Adder)	6
1.4 先行进位加法器 (Carry-Lookahead Adder)	6
1.5 前缀加法器 (Prefix Adder)	7
1.6 三种加法器延迟对比	7
1.7 减法器 (Subtractor)	7
1.8 比较器 (Comparator)	8
2 ALU 算术逻辑单元 & 状态标志位	9
2.1 ALU 基本功能	9
2.2 ALU 四标志位详解	9
2.2.1 N (Negative) 标志	9
2.2.2 Z (Zero) 标志	9
2.2.3 C (Carry) 标志	10
2.2.4 V (oVerflow) 标志——考试重中之重	10
2.3 ALU 标志位例题	11
2.4 ALU SystemVerilog 代码	11
3 移位器 (Shifters)	13
3.1 三种移位器对比	13
3.2 详细示例 (5-bit: 11001)	13
3.2.1 逻辑移位 (Logical Shift) 要点	13
3.2.2 算术移位 (Arithmetic Shift) 要点	14
3.2.3 循环移位 (Rotate) 要点	14
3.3 移位器作为乘法器/除法器	14
3.4 移位器设计 (桶形移位器)	14
4 乘法器与除法器	15
4.1 乘法器 (Multiplier)	15
4.2 除法器 (Divider)	15

5 数制系统 (Number Systems)	16
5.1 定点数 (Fixed-Point Numbers)	16
5.2 浮点数 (Floating-Point Numbers)	16
6 IEEE 754 浮点数格式	16
6.1 IEEE 754 单精度 (32-bit) 格式	16
6.2 隐含前导 1 原理 (Implicit Leading 1)	17
6.3 IEEE 754 特殊值	17
6.4 IEEE 754 双精度 (64-bit)	17
6.5 IEEE 754 转换步骤	17
6.6 IEEE 754 必考例题	18
6.6.1 例题 1: 228 转 IEEE 754 单精度	18
6.6.2 例题 2 (必考): -58.25 转 IEEE 754 单精度	18
6.6.3 例题 3: 浮点加法—— $0x3FC00000 + 0x40500000$	19
6.7 浮点舍入模式	20
7 计数器与移位寄存器	21
7.1 计数器 (Counter)	21
7.2 移位寄存器 (Shift Register)	21
7.3 带并行加载的移位寄存器	21
8 存储器阵列 (Memory Arrays)	22
8.1 基本概念	22
8.2 存储类型对比	22
8.3 DRAM (动态随机存取存储器)	22
8.4 SRAM (静态随机存取存储器)	22
8.5 ROM (只读存储器)	23
8.6 存储器实现组合逻辑——查找表 (LUT)	23
8.7 多端口存储器 (Multi-ported Memory)	23
8.8 SystemVerilog 存储器示例	23
9 逻辑阵列 (Logic Arrays)	24
9.1 PLA (可编程逻辑阵列)	24
9.2 FPGA (现场可编程门阵列)	24
9.3 FPGA 设计流程	24
9.4 PLA vs FPGA 对比	25
10 考试重点提示	26
10.1 第一重点: ALU 标志位 N/Z/C/V (★★★★)	26

10.2 第二重点：移位器三种类型 (★★★★)	26
10.3 第三重点：IEEE 754 浮点数 (★★★★)	26
10.4 其他考点	27

加法器 (Adders)

1 位加法器 (1-Bit Adders)

半加器 (Half Adder)

半加器接受两个 1 位输入 A 和 B，产生和 S 与进位 C_{out} 。

A	B	S	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

逻辑表达式：

$$S = A \oplus B, \quad C_{out} = AB \quad (1)$$

全加器 (Full Adder)

全加器在半加器基础上增加进位输入 C_{in} 。

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

逻辑表达式：

$$S = A \oplus B \oplus C_{in}, \quad C_{out} = AB + AC_{in} + BC_{in} \quad (2)$$

多比特加法器——进位传播加法器 (CPA)

多比特加法器 (Carry Propagate Adder, CPA) 有三种类型：

1. 行波进位加法器 (Ripple-Carry Adder) ——最慢
2. 先行进位加法器 (Carry-Lookahead Adder, CLA) ——快
3. 前缀加法器 (Prefix Adder) ——最快

行波进位加法器 (Ripple-Carry Adder)

将 N 个 1 位全加器串联，进位依次”波纹”传播。

延迟：

$$t_{\text{ripple}} = N \cdot t_{\text{FA}} \quad (3)$$

其中 t_{FA} 为单个全加器延迟。延迟为 $O(N)$ 。

缺点：速度慢，进位必须依次传播通过所有位。

先行进位加法器 (Carry-Lookahead Adder)

将加法器分为 k 位一组，通过产生/传播信号提前计算进位，避免逐位传播。

关键定义：

- 第 i 列的**产生 (Generate)** 信号： $G_i = A_i B_i$ [当 A_i 和 B_i 都为 1 时，该列”产生”进位]
- 第 i 列的**传播 (Propagate)** 信号： $P_i = A_i \oplus B_i$ [当 A_i 或 B_i 为 1 时，该列”传播”进位]
- 第 i 列的**进位输出**： $C_i = A_i B_i + (A_i \oplus B_i) C_{i-1} = G_i + P_i C_{i-1}$

4 位块传播/产生信号：

$$P_{3:0} = P_3 P_2 P_1 P_0 \quad (4)$$

$$G_{3:0} = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0)) \quad (5)$$

$$C_i = G_{i:j} + P_{i:j} C_{j-1} \quad (6)$$

CLA 加法步骤 (32 位 + 4 位块)：

1. **Step 1:** 计算所有列的 G_i 和 P_i
2. **Step 2:** 计算 k -bit 块的 G 和 P
3. **Step 3:** C_{in} 通过块传播/产生逻辑传播 (同时计算部分和)
4. **Step 4:** 计算最高位块的和

延迟 (N 位 CLA, k 位块)：

$$t_{\text{CLA}} = t_{pg} + t_{pg_block} + \left(\frac{N}{k} - 1 \right) t_{AND_OR} + k \cdot t_{FA} \quad (7)$$

对于 $N > 16$, CLA 明显快于行波进位。延迟为 $O(\log N)$ 。

前缀加法器 (Prefix Adder)

计算每一列的进位输入 C_{i-1} ，然后计算和。

核心思想：

- 进位既可以在某一列产生，也可以从前面列传播而来
- 第 -1 列（即 C_{in} ）： $G_{-1} = C_{in}$, $P_{-1} = 0$
- 第 i 列的进位输入 = 第 $i-1$ 列的进位输出： $C_{i-1} = G_{i-1:-1}$
- 和公式： $S_i = (A_i \oplus B_i) \oplus G_{i-1:-1}$

块合并公式：

$$G_{i:j} = G_{i:k} + P_{i:k} G_{k-1:j} \quad (8)$$

$$P_{i:j} = P_{i:k} P_{k-1:j} \quad (9)$$

延迟（N 位前缀加法器）：

$$t_{PA} = t_{pg} + \log_2 N \cdot t_{pg_prefix} + t_{XOR} \quad (10)$$

延迟为 **O(log N)**，但常数因子比 CLA 更小。

三种加法器延迟对比

加法器类型	延迟公式	复杂度	32 位延迟
行波进位 (Ripple-Carry)	$N \cdot t_{FA}$	O(N)	9.6 ns
先行进位 (CLA, 4-bit 块)	$t_{pg} + t_{pg_block} + (\frac{N}{k} - 1)t_{AND_OR} + k \cdot t_{FA}$	O(log N)	3.3 ns
前缀 (Prefix)	$t_{pg} + \log_2 N \cdot t_{pg_prefix} + t_{XOR}$	O(log N)	1.2 ns

假设：2 输入门延迟 = 100 ps；全加器延迟 = 300 ps。

结论：前缀加法器最快（1.2 ns），行波进位最慢（9.6 ns），CLA 居中（3.3 ns）。行波进位尽管最慢，但硬件开销最小。

减法器 (Subtractor)

利用二进制补码原理： $Y = A - B = A + \overline{B} + 1$ 。在 B 输入端加反相器，并将 C_{in} 设为 1 即可实现。

比较器 (Comparator)

- 相等比较器：XNOR 每个对应位，然后 AND 所有结果
- 小于比较器： $A < B$ 当且仅当 $A - B$ 的最高位（符号位）为 1

ALU 算术逻辑单元 & 状态标志位

任老师原话：“曾经是考试题，下次还会考”

ALU 基本功能

ALU 执行四种基本操作，由 ALUControl[1:0] 控制：

ALUControl[1:0]	功能
00	Add（加法）
01	Subtract（减法）
10	AND（按位与）
11	OR（按位或）

ALU 四标志位详解

考试核心：N、Z、C、V 四个状态标志位的含义和设置规则。

标志	名称	条件	含义
N	Negative	Result 的最高位 = 1	结果为负数（补码表示）
Z	Zero	Result 所有位 = 0	结果为零
C	Carry	加法器的 $C_{out} = 1$	加法器产生进位输出（仅当 ALU 做加减法时有效）
V	oVerflow	见下方详细规则	有符号数运算溢出

N (Negative) 标志

$$N = Result[31] \text{ (最高位, 即符号位)} \quad (11)$$

- N = 1：结果为负（补码表示中最高位为 1）
- N 直接连接 Result 的最高有效位 (MSB)

Z (Zero) 标志

$$Z = 1 \iff Result = 0 \quad (12)$$

- Z = 1：结果全零
- 硬件实现：对 Result 所有位做 NOR（缩减或非运算）

C (Carry) 标志

$$C = 1 \iff ALU \ / \ C_{out} = 1 \quad (13)$$

- $C = 1$: ALU 在执行 AND/OR 时 C 无效（无意义）
- C 仅在 $ALUControl[1] = 0$ （即加或减）时才有意义
- 硬件实现: $C = ALUControl[1]' \cdot C_{out_adder}$

V (oVerflow) 标志——考试重中之重

溢出判定规则:

溢出 ($V=1$) 当且仅当:

1. ALU 在执行加法或减法, 并且
2. A 和 Sum 的符号位不同 (即结果符号与操作数 A 符号相反)

这等价于:

- 加法时: A 和 B 同号, 但结果与它们异号 [正 + 正 = 负或负 + 负 = 正]
- 减法时: A 和 B 异号, 但结果与 A 异号 [正-负 = 负或负-正 = 正]

硬件公式 (SystemVerilog):

$$overflow = (ALUControl[1] = 0) \& \sim (a_{msb} \oplus b_{msb} \oplus ALUControl[0]) \& (a_{msb} \oplus sum_{msb}) \quad (14)$$

通俗理解:

- 两个正数相加得到负数 → 溢出
- 两个负数相加得到正数 → 溢出
- 正数减负数得到负数 → 溢出
- 负数减正数得到正数 → 溢出
- 一正一负相加 不会溢出

ALU 标志位例题

例题 1：设 4-bit ALU， $A=0101$ (5), $B=0011$ (3)，执行 $A+B$ 。求 N 、 Z 、 C 、 V 。

解：

$$\text{Result} = 0101 + 0011 = 1000 \quad (15)$$

$$N = 1 \quad (\text{最高位为 } 1) \quad (16)$$

$$Z = 0 \quad (\text{不全为 } 0) \quad (17)$$

$$C = 0 \quad (4 \text{ 位加法无进位输出}) \quad (18)$$

$$V = 1 \quad (5+3=8 > 7 \text{ 溢出! 正} + \text{正} = \text{负}) \quad (19)$$

例题 2：设 4-bit ALU， $A=1100$ (-4), $B=1010$ (-6)，执行 $A+B$ 。求 N 、 Z 、 C 、 V 。

解：

$$\text{Result} = 1100 + 1010 = (1)0110 \quad \text{进位 } 1, \text{ 结果 } 0110 \quad (20)$$

$$N = 0 \quad (\text{最高位为 } 0) \quad (21)$$

$$Z = 0 \quad (22)$$

$$C = 1 \quad (\text{有进位输出}) \quad (23)$$

$$V = 1 \quad (-4)+(-6)=-10 < -8 \text{ 溢出! 负} + \text{负} = \text{正}) \quad (24)$$

例题 3：设 4-bit ALU， $A=0111$ (7), $B=0010$ (2)，执行 $A-B$ 。求 N 、 Z 、 C 、 V 。

解：

$$\text{Result} = 0111 - 0010 = 0111 + 1110 = (1)0101 \quad \text{结果为 } 0101=5 \quad (25)$$

$$N = 0 \quad (26)$$

$$Z = 0 \quad (27)$$

$$C = 1 \quad (\text{减法通过加补码实现, 有进位}) \quad (28)$$

$$V = 0 \quad (7-2=5 \text{ 在 } -8 \sim 7 \text{ 范围内, 无溢出}) \quad (29)$$

ALU SystemVerilog 代码

```
module alu #(parameter WIDTH = 31)(input logic [WIDTH:0] a, b,
    input logic [1:0] ALUControl,
    output logic [WIDTH:0] Result,
    output logic [3:0] ALUFlags);    // {N, Z, C, V}

    logic neg, zero, carry, overflow;
    logic [WIDTH:0] condinvb;
```

```
logic [WIDTH+1:0] sum;

assign condinvb = ALUControl[0] ? ~b : b;
assign sum = a + condinvb + ALUControl[0];

always_comb
    casex (ALUControl[1:0])
        2'b0?: Result = sum;
        2'b10: Result = a & b;
        2'b11: Result = a | b;
    endcase

assign neg      = Result[WIDTH];           // N flag
assign zero     = (Result == (WIDTH+1)'b0); // Z flag
assign carry    = (ALUControl[1]==0) & sum[WIDTH+1]; // C flag
assign overflow = (ALUControl[1]==0) &
    ~(a[WIDTH]^b[WIDTH]^ALUControl[0]) &
    (a[WIDTH]^sum[WIDTH]);                // V flag
assign ALUFlags = {neg, zero, carry, overflow};
endmodule
```

移位器 (Shifters)

任老师重点：三种移位器类型必须区分清楚

三种移位器对比

移位类型	操作	空位填充规则	典型用途
逻辑移位	左移 ($\ll$) / 右移 ($\gg$)	始终填 0	无符号数移位；位操作
算术移位	左移 ($\ll<$) / 右移 ($\gg>$)	右移: 填旧 MSB 左移: 填 0	有符号数乘除；保持符号
循环移位	右旋 (ROR) / 左旋 (ROL)	移出的位从另一端进入	位重排；密码学

详细示例 (5-bit: 11001)

类型	操作	结果 (5-bit)	说明
逻辑	11001 $\gg$ 2	00110	右移 2 位，左端填 0
	11001 $\ll$ 2	00100	左移 2 位，右端填 0
算术	11001 $\gg>$ 2	11110	右移 2 位，左端填旧 MSB=1
	11001 $\ll<$ 2	00100	左移 2 位，右端填 0
循环	11001 ROR 2	01110	右旋 2 位，低 2 位 (01) 移到高端
	11001 ROL 2	00111	左旋 2 位，高 2 位 (11) 移到低端

逻辑移位 (Logical Shift) 要点

- 左移/右移 空位一律填 0
- 不关心符号，当作无符号数处理
- 左移 = 乘以 2^N (对于无符号数)
- 右移 = 除以 2^N (对于无符号数)

算术移位 (Arithmetic Shift) 要点

- 算术左移：空位填 0（与逻辑左移相同）
- 算术右移：空位填旧的最高有效位 (MSB)
 - 正数 (MSB=0)：右移填 0 → 与逻辑右移相同
 - 负数 (MSB=1)：右移填 1 → 保持负号（补码算术右移）
- 算术右移 = 有符号数除以 2^N （向下取整）

循环移位 (Rotate) 要点

- 移出的位环绕到另一端
- ROR (Rotate Right): 低位移出，从高端进入
- ROL (Rotate Left): 高位移出，从低端进入
- 位总数不变，只是重新排列

移位器作为乘法器/除法器

- $A \ll N = A \times 2^N$
 - 例：00001 $\ll$ 2 = 00100 ($1 \times 4 = 4$)
 - 例：11101 $\ll$ 2 = 10100 ($-3 \times 4 = -12$)
- $A \ggg N = A \div 2^N$ （算术右移用于有符号除法）
 - 例：01000 $\ggg$ 2 = 00010 ($8 \div 4 = 2$)
 - 例：10000 $\ggg$ 2 = 11100 ($-16 \div 4 = -4$)

移位器设计（桶形移位器）

使用多级 4:1 多路复用器实现：第 0 级按 1-bit 移位，第 1 级按 2-bit 移位……最终形成对数深度的桶形移位器。

乘法器与除法器

乘法器 (Multiplier)

乘法通过部分积 (partial products) 累加实现。

二进制乘法示例: $5 \times 7 = 35$

十进制	二进制
	0101
230	× 0111
× 42	0101
460	0101
+ 920	0101
9660	+ 0000
	0100011

硬件实现: $N \times N$ 乘法器需要 N^2 个与门产生部分积, 然后通过加法器树求和, 最终输出 $2N$ 位结果。

除法器 (Divider)

除法算法 (恢复余数法):

```

R' = 0
for i = N-1 downto 0:
    R = {R' << 1, A[i]}    // 左移R', 并加入A的当前位
    D = R - B
    if D < 0:
        Q[i] = 0
        R' = R              // 恢复
    else:
        Q[i] = 1
        R' = D
R = R'
  
```

二进制示例: $1101 \div 10 = 0110 \text{ R}1$ (即 $13 \div 2 = 6 \text{ 余} 1$)

数制系统 (Number Systems)

定点数 (Fixed-Point Numbers)

二进制小数点位置固定。

示例：用 4 整数位 + 4 小数位表示 6.75

$$6.75_{10} = 0110.1100_2 = 2^2 + 2^1 + 2^{-1} + 2^{-2} = 4 + 2 + 0.5 + 0.25 = 6.75$$

有符号定点数可以用原码 (Sign/Magnitude) 或补码 (Two's complement) 表示。

示例：用 4 整数位 + 4 小数位表示 -7.5

- 原码：11111000 (符号位 1, 数值 7.5=01111000)
- 补码：10001000 (+7.5=01111000, 取反 =10000111, +1=10001000)

浮点数 (Floating-Point Numbers)

二进制小数点浮在最显著 1 的右边，类似科学记数法。

$$\pm M \times B^E$$

- M = mantissa (尾数)
- B = base (基数，二进制为 2)
- E = exponent (指数)

IEEE 754 浮点数格式

任老师原话：“等于送分给你们”

IEEE 754 单精度 (32-bit) 格式

符号位 Sign	阶码 Exponent	尾数 Fraction / Mantissa
1 bit	8 bits	23 bits
S	E (biased)	F

- 符号位 S: 1 bit。0 = 正数, 1 = 负数
- 阶码 E: 8 bits。使用偏置 127 (biased by 127)
 - 实际指数 = E - 127 (偏置指数 = bias + 实际指数 = 127 + 实际指数)

– 范围：-126 到 +127 (E=0 和 E=255 为特殊值保留)

- 尾数 F：23 bits。存储小数部分，整数部分 1 为隐含前导 1 (implicit leading 1)

隐含前导 1 原理 (Implicit Leading 1)

二进制科学记数法中，规格化后尾数的整数位总是 1：

$$1.\underbrace{xxxxx\dots}_{23 \text{ bits}}$$

因此不需要存储这个 1，只存储 23 位小数部分。这使实际精度达到 24 位。

IEEE 754 特殊值

数值	符号位 S	阶码 E	尾数 F
0	X	00000000	全 0
∞	0	11111111	全 0
$-\infty$	1	11111111	全 0
NaN	X	11111111	非零
Denormalized	X	00000000	非零

IEEE 754 双精度 (64-bit)

字段	位数	说明
符号位 S	1 bit	0 正 1 负
阶码 E	11 bits	bias = 1023
尾数 F	52 bits	隐含前导 1

IEEE 754 转换步骤

十进制 $\rightarrow$ IEEE 754 单精度：

1. 十进制转二进制：分别转换整数部分和小数部分

- 整数部分：连续除以 2，取余数（倒序）
- 小数部分：连续乘以 2，取整数部分（正序）

2. 二进制科学记数法：写成 $1.xxx \times 2^E$ 的形式（规格化）

3. 填入各字段：

- $S = 0$ (正) 或 1 (负)
- E (偏置) = 实际指数 + 127, 转为 8 位二进制
- F = 尾数小数部分 23 位 (省去前导 1)

4. 组装结果: $S | E(8 \text{ 位}) | F(23 \text{ 位}) \rightarrow$ 可选转十六进制

IEEE 754 必考例题

例题 1: 228 转 IEEE 754 单精度

1. 十进制转二进制: $228_{10} = 11100100_2$
2. 二进制科学记数法: $11100100_2 = 1.11001_2 \times 2^7$
3. 填入字段:

- $S = 0$ (正数)
- 偏置指数 = $127 + 7 = 134 = 10000110_2$
- $F = 110 \ 0100 \ 0000 \ 0000 \ 0000 \ 0000$ (23 位, 前导 1 不存)

S	E	F
0	10000110	110 0100 0000 0000 0000 0000

十六进制: **0x43640000**

例题 2 (必考): -58.25 转 IEEE 754 单精度

1. 十进制转二进制:

- 整数部分 58:
 - $58 \div 2 = 29$ 余 0
 - $29 \div 2 = 14$ 余 1
 - $14 \div 2 = 7$ 余 0
 - $7 \div 2 = 3$ 余 1
 - $3 \div 2 = 1$ 余 1
 - $1 \div 2 = 0$ 余 1

$$58_{10} = 111010_2$$

- 小数部分 0.25:
 - $0.25 \times 2 = 0.5 \rightarrow$ 整数 0

$$- 0.5 \times 2 = 1.0 \rightarrow \text{整数 } 1$$

$$0.25_{10} = .01_2$$

$$\bullet \text{ 合起来: } 58.25_{10} = 111010.01_2$$

$$2. \text{ 二进制科学记数法: } 111010.01_2 = 1.1101001_2 \times 2^5$$

3. 填入字段:

- $S = 1$ (负数)
- 偏置指数 $= 127 + 5 = 132 = 10000100_2$
- $F = 110\ 1001\ 0000\ 0000\ 0000\ 0000$ (前导 1 不存, 右侧补零)

S	E	F
1	10000100	110 1001 0000 0000 0000 0000

十六进制: **0xC2690000**

例题 3: 浮点加法——**0x3FC00000 + 0x40500000**

Step 1: 提取字段

- $N1 = 0x3FC00000$
 - $S=0, E=01111111=127, F=100\ 0000\dots=0.5$
 - 加前导 1: $\text{mantissa} = 1.5 = 1.1_2$
- $N2 = 0x40500000$
 - $S=0, E=10000000=128, F=101\ 0000\dots=0.625$
 - 加前导 1: $\text{mantissa} = 1.625 = 1.101_2$

Step 2: 对齐指数 (向大的对齐)

- $E1=127, E2=128, \text{差} = -1$
- $N1$ 的尾数右移 1 位: $1.1 \gg 1 = 0.11$ (指数对齐到 128)

Step 3: 尾数相加

$$0.11 \times 2^{128} + 1.101 \times 2^{128} = 10.011 \times 2^{128}$$

Step 4: 规格化

$$10.011 \times 2^{128} = 1.0011 \times 2^{129}$$

Step 5: 组装结果

- $S = 0, E = 129 = 10000001, F = 0011\dots$
- 结果 $= 0x40980000$

浮点舍入模式

- 向负无穷 (Down): $1.100101 \rightarrow 1.100$
- 向正无穷 (Up): $1.100101 \rightarrow 1.101$
- 向零 (Toward zero): $1.100101 \rightarrow 1.100$
- 向最近 (To nearest): $1.100101 \rightarrow 1.101$ (默认模式, 1.625 比 1.5 更接近 1.578125)

上溢 (Overflow): 数值过大, 超出表示范围。 下溢 (Underflow): 数值过小, 超出表示范围。

计数器与移位寄存器

计数器 (Counter)

每个时钟沿计数值 +1，用于循环遍历数值。例如：000, 001, 010, 011, 100, 101, 110, 111, 000, 001...

典型应用：

- 数字钟表显示
- 程序计数器 (PC)：跟踪当前正在执行的指令

实现：累加器 + 寄存器的组合。Reset 信号用来复位到 0。

移位寄存器 (Shift Register)

每个时钟沿：

- 移入一个新位
- 移出一个位

应用：

- 串行-并行转换器 (Serial-to-Parallel)： $S_{in} \rightarrow Q_{0:N-1}$
- 并行-串行转换器 (Parallel-to-Serial)： $D_{0:N-1} \rightarrow S_{out}$ [需要带并行加载功能的移位寄存器：Load=1 时作为普通寄存器，Load=0 时移位]

带并行加载的移位寄存器

- Load = 1：作为普通 N 位寄存器（并行加载 $D_{0:N-1}$ ）
- Load = 0：作为移位寄存器（ $S_{in} \rightarrow Q_0 \rightarrow Q_1 \rightarrow \cdots \rightarrow S_{out}$ ）

存储器阵列 (Memory Arrays)

基本概念

参数	含义
深度 (Depth)	行数 = 字数 = 2^N (N 位地址)
宽度 (Width)	列数 = 每字位数 = M
容量 (Size)	$\times = 2^N \times M$

示例： $2^2 \times 3$ 阵列 = 4 字 $\times$ 3 位。地址 10 存储字 100。

存储类型对比

类型	易失性	存储元件	特点	典型用途
DRAM	易失 (volatile)	电容	需刷新；密度高；慢	主内存
SRAM	易失 (volatile)	交叉耦合反相器	不需刷新；速度快；贵	缓存 (Cache)
ROM / Flash	非易失 (non-volatile)	浮栅晶体管等	断电保持数据；写入慢/不可写	固件；U 盘；SSD

DRAM (动态随机存取存储器)

- 存储原理：用电容上的电荷表示数据
- ”动态”含义：电容电荷会泄漏，需周期性刷新（重写）• 读取操作会破坏存储值
- 优点：每位只需 1 个晶体管 + 1 个电容，密度高、成本低
- 缺点：需要刷新电路，速度较慢

SRAM (静态随机存取存储器)

- 存储原理：交叉耦合反相器形成双稳态电路
- ”静态”含义：只要供电就保持数据，无需刷新
- 优点：速度快，无需刷新
- 缺点：每位需 6 个晶体管，密度低、成本高

ROM (只读存储器)

- 非易失：断电后数据不丢失
- 点标记法 (Dot Notation)：有位线连接 = 存储 1；无连接 = 存储 0
- ROM 也可用来实现组合逻辑：将地址当作输入，数据当作输出（查表法）

存储器实现组合逻辑——查找表 (LUT)

使用 $2^N \times 1$ 存储器实现 N 输入逻辑函数：将真值表的输出值存储在对应地址。

示例：用 $2^2 \times 3$ ROM 实现：

$$X = AB \quad (30)$$

$$Y = A + B \quad (31)$$

$$Z = A \oplus B \quad (32)$$

多端口存储器 (Multi-ported Memory)

- 端口 (Port)：一个地址/数据对
- 典型的寄存器文件 (Register File) 有 3 个端口：2 读 + 1 写
- 用于 CPU 寄存器文件：同时读取两个源操作数，写入一个结果

SystemVerilog 存储器示例

```
module dmem(input logic clk, we,
    input logic [7:0] a,
    input logic [2:0] wd,
    output logic [2:0] rd);
    logic [2:0] RAM[255:0];
    assign rd = RAM[a];
    always @(posedge clk)
        if (we) RAM[a] <= wd;
endmodule
```

逻辑阵列 (Logic Arrays)

PLA (可编程逻辑阵列)

结构：AND 阵列 + OR 阵列，仅实现组合逻辑。

组成部分	说明
AND 阵列	产生乘积项 (implicants / product terms)
OR 阵列	将乘积项求和

特点：内部连接固定（通过熔丝编程），适合实现 SOP 形式的逻辑函数。

示例：实现 $X = ABC + \overline{A}BC$, $Y = \overline{A}B$

FPGA (现场可编程门阵列)

FPGA 组成部分：

1. **LE (Logic Element)/逻辑单元**：执行逻辑功能
 - LUT (查找表)：组合逻辑
 - 触发器 (Flip-flop)：时序逻辑
 - 多路复用器：连接 LUT 和触发器
2. **IOE (I/O Element)**：与外部世界接口
3. **可编程互连 (Programmable Interconnection)**：连接 LE 与 IOE
4. **其他模块**：部分 FPGA 包含硬核乘法器、RAM 块等

典型 LE 结构 (Altera Cyclone IV)：

- 1 个 4 输入 LUT
- 1 个寄存器输出
- 1 个组合逻辑输出

FPGA 设计流程

1. 设计输入（原理图或 HDL）
2. 仿真验证
3. 综合并映射到 FPGA

4. 下载配置到 FPGA

5. 测试验证

PLA vs FPGA 对比

特性	PLA	FPGA
逻辑功能	仅组合逻辑	组合 + 时序逻辑
内部连接	固定（一次性编程）	可编程（可重配置）
结构	AND 阵列 + OR 阵列	LE 阵列 + 可编程互连
灵活性	低	高

考试重点提示

任胜兵老师考试重点——必考内容总结

第一重点：ALU 标志位 N/Z/C/V (★★★★)

1. **N (Negative)**: 结果最高位。 $N = \text{Result}[\text{MSB}]$ 。 [考法: 给定运算, 判断 N]
2. **Z (Zero)**: 结果全零。 $Z = (\text{Result} == 0)$ 。 [考法: 判断运算结果是否为 0]
3. **C (Carry)**: 加法器进位输出。 $C = C_{out}$ (仅加法/减法时有效)。 [考法: 判断进位]
4. **V (oVerflow)**: 有符号溢出。
 - 加法溢出: 同号相加得异号
 - 减法溢出: 异号相减得与 A 异号

[考法: 给定两个数做加减, 判断是否溢出。必须会算!]

第二重点：移位器三种类型 (★★★★)

类型	右移空位填	左移空位填	特色
逻辑移位	0	0	无符号, 简单
算术移位	旧 MSB (符号扩展)	0	有符号, 保持符号
循环移位	移出位的环绕	移出位的环绕	位旋转

记忆诀窍: 算术移位只关心右移 (填 MSB 保持符号), 左移和逻辑移位一样填 0。循环移位就是”转圈圈”。

第三重点：IEEE 754 浮点数 (★★★)

- 格式 (必背): 1 bit 符号 + 8 bit 偏置阶码 + 23 bit 尾数
- 偏置 (Bias): 127 (单精度)、1023 (双精度)
- 隐含前导 1: 规格化尾数形如 $1.xxx$, 1 不存储
- 转换步骤: 十进制 $\rightarrow$ 二进制 $\rightarrow$ 科学记数法 $\rightarrow$ 填字段
- 例题必会: -58.25 转 IEEE 754 = 0xC2690000
- 特殊值: 全 0 阶码 + 全 0 尾数 = 0; 全 1 阶码 + 全 0 尾数 = ∞ ; 全 1 阶码 + 非 0 尾数 = NaN

其他考点

- 加法器延迟对比：行波 $O(N)$ > CLA $O(\log N)$ > 前缀 $O(\log N)$ ，前缀最快
- 全加器逻辑： $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + AC_{in} + BC_{in}$
- CLA 产生/传播信号： $G_i = A_i B_i$, $P_i = A_i \oplus B_i$, $C_i = G_i + P_i C_{i-1}$
- DRAM vs SRAM: DRAM 用电容 (需刷新、密度高), SRAM 用反相器 (快、贵)
- ROM 也可实现组合逻辑：查表法 (LUT)，地址 = 输入，数据 = 输出
- PLA vs FPGA: PLA 仅组合逻辑，FPGA 含组合 + 时序 + 可编程互连

祝考试顺利！